

Magentic: governed assistants inside real workflows

Magentic is a local workspace where a team trains an assistant on its own content, approves it the way it would approve a release, and then puts it to work inside the team's delivery workflow. It runs on a laptop with a local model, and everything it exposes is available over MCP, so it plugs into the tools a team already has.

THE RULE THAT MAKES IT SAFE TO USE

An assistant is a signed release plus its own instructions. It can only act while that release is approved by two distinct reviewers. Retire the release and chat, the workflow scheduler, the editor plugin and any MCP client stop using it at the same moment, because every one of them asks the same function the same question: can this assistant work right now?

WHAT THE DEMO SHOWS

A fictional consulting practice, Northwind Federal, delivering Salesforce work for federal agencies.

1. **Training portal.** The practice's engagement notes (synthetic, fictional agencies) become a dataset. Secrets are counted and never shown. The readiness suite runs, a release is requested, two reviewers sign, and the assistant is created with the practice's own wording: name the agency and the org, numbered steps, name the control or ATO evidence touched, end with "Next step:".
2. **Delivery workflow.** Intake → Discovery → Solution design → Build → QA → Client UAT → Go live, with review gates before design, UAT and go live. The assistant staffs Discovery and QA.
3. **A live engagement.** A grant inquiry routing request for a fictional bureau is started. Within seconds the assistant drafts Discovery in the practice's voice. A person completes the stage with the draft or writes their own. The owner cannot approve their own work; two reviewers sign the design gate. Every transition becomes a Jira preview; delivery to Jira is a deliberate admin action.
4. **Cost and control.** Each draft shows what it cost in tokens; the home page sums cost per assistant. Retire the release and the next engagement is refused at Discovery, with the stage named.

WHAT IT IS BUILT ON

- Local application: one server, one browser app, a VS Code extension, two MCP endpoints (workspace and pipeline). Data is one JSON document per workspace with atomic writes; credentials never enter records or logs.
- Models: local Ollama by default; OpenAI, Anthropic, Gemini, Azure OpenAI, or any OpenAI compatible gateway configured on the server, never from the browser.
- Test first: the Salesforce delivery and software delivery scenarios exist as executable specifications (unit and browser tests) before they exist as pages.

WHERE IT IS GOING

- A documentation portal: drop agency documentation into the workspace and attach it to a run, add it to an assistant's dataset, or both, with the same review rules.
- Data platform stages: a workflow stage that reads from a governed data source through MCP.
- Marketplace: share the shape of an assistant (recipe, instructions, evaluation suite, workflow template) without sharing the data.

CONTACT

Nihal Sirivolu · CMDA, Virginia Tech, May 2027 · sirivolun@gmail.com